Casos de uso del sistema Telegram Bot

Proyecto académico de bot conversacional con portal web, automatización e integraciones

Alcance del sistema

El sistema Telegram Bot permite que un usuario se registre desde una página web, verifique su cuenta de Telegram, autorice aplicaciones externas y luego converse con el bot desde Telegram. El proyecto toma como referencia el bot de WhatsApp construido previamente, pero se enfoca en una versión para Telegram con despliegue en Railway, base de datos Neon PostgreSQL e integraciones externas como Google Calendar, GitHub y YouTube.

La página web funciona como punto de entrada: allí el usuario inicia sesión con Telegram o confirma su número mediante un enlace hacia Telegram, concede permisos a las aplicaciones conectadas y finalmente abre el chat del bot. Las instrucciones encontradas dentro de documentos de referencia del profesor se usaron únicamente como ejemplo de estructura académica. Los casos de uso siguientes son propios del proyecto Telegram Bot.

Actores principales

- Usuario: persona que entra a la página web, verifica su cuenta de Telegram, autoriza integraciones y conversa con el bot.
- Administrador: persona responsable de configurar el bot, gestionar permisos, revisar errores y conectar servicios externos.
- Sistema externo: API o plataforma conectada al bot, por ejemplo Telegram, Google Calendar, GitHub, YouTube, Groq, Railway o Neon.

Casos de uso

CU01: Registro y verificación desde la página web

Actor:	Usuario
Descripción:	El usuario entra a la página del proyecto para registrarse, verificar su identidad de Telegram y dejar lista la cuenta antes de hablar con el bot.
Precondición:	La página web debe estar publicada y el bot debe existir en Telegram.
Flujo principal:	1. El usuario abre la página web del Telegram Bot. 2. El sistema muestra la opción de iniciar sesión con Telegram o abrir un enlace de verificación. 3. El usuario confirma su identidad de Telegram mediante el mecanismo disponible. 4. El sistema valida el identificador recibido y crea o actualiza la cuenta del usuario. 5. La página muestra el estado de verificación y el botón para continuar al bot.
Postcondición:	El usuario queda registrado y asociado a una cuenta de Telegram válida.
Flujo alternativo:	Si el usuario no completa la verificación, el sistema no habilita las integraciones y muestra una opción para intentarlo de nuevo.

CU02: Autorizar aplicaciones conectadas

Actor:	Usuario
Descripción:	El usuario concede permisos para que el bot pueda usar servicios externos como Google Calendar, GitHub o YouTube desde su cuenta.
Precondición:	El usuario debe estar verificado en la página web.
Flujo principal:	1. El usuario selecciona la aplicación que desea conectar. 2. El sistema redirige al flujo de autorización del servicio externo. 3. El usuario acepta los permisos solicitados. 4. El sistema recibe la confirmación y guarda los tokens de forma segura.

	5. La página muestra qué aplicaciones quedaron conectadas.
Postcondición:	El bot queda autorizado para ejecutar acciones permitidas en nombre del usuario.
Flujo alternativo:	Si el usuario rechaza permisos o el servicio falla, la aplicación queda como no conectada y el bot informa que esa función no está disponible.

CU03: Inicio y consulta del menú en Telegram

Actor:	Usuario
Descripción:	El usuario abre el chat del bot desde la página o directamente desde Telegram y solicita la lista de comandos disponibles.
Precondición:	El bot debe estar encendido y conectado a Telegram; idealmente el usuario ya debe estar verificado.
Flujo principal:	1. El usuario pulsa el botón para ir a Telegram o escribe /start, /menu o /ayuda. 2. El bot recibe el mensaje y valida el chat del usuario. 3. El sistema revisa si el usuario está registrado y qué integraciones tiene activas. 4. El bot genera un menú personalizado con las funciones disponibles. 5. El bot envía la respuesta al chat del usuario.
Postcondición:	El usuario conoce las opciones disponibles para interactuar con el sistema.
Flujo alternativo:	Si el usuario no está verificado, el bot envía el enlace de la página para completar el registro.

CU04: Conversación con inteligencia artificial

Actor:	Usuario
Descripción:	El usuario envía un mensaje libre y el bot responde usando un modelo de IA, conservando contexto básico de la conversación.
Precondición:	El bot debe tener configurada la llave de la API de IA y conexión a internet.

Flujo principal:	1. El usuario envía una pregunta o mensaje que no corresponde a un comando. 2. El bot guarda el mensaje en la memoria temporal o persistente del chat. 3. El sistema envía el historial reciente al servicio de IA. 4. La IA genera una respuesta corta en español. 5. El bot envía la respuesta al usuario y actualiza la memoria.
Postcondición:	El usuario recibe una respuesta generada por IA y la conversación conserva continuidad.
Flujo alternativo:	Si la API de IA falla o no tiene llave configurada, el bot responde con un mensaje de error controlado.

CU05: Uso de juegos y utilidades básicas

Actor:	Usuario
Descripción:	El usuario ejecuta comandos como trivia, dado, moneda, piedra papel tijera, hora, eco, suma de pares o suma de dígitos.
Precondición:	El bot debe estar activo y el comando debe escribirse con el formato esperado.
Flujo principal:	1. El usuario escribe un comando de juego o utilidad. 2. El bot identifica el comando y extrae los parámetros necesarios. 3. El sistema ejecuta la lógica solicitada. 4. El bot responde con el resultado en el chat. 5. Si aplica, el sistema actualiza el puntaje o estado del juego.
Postcondición:	El usuario obtiene el resultado del juego o utilidad solicitada.
Flujo alternativo:	Si faltan parámetros o el formato es incorrecto, el bot muestra el uso correcto del comando.

CU06: Crear evento en Google Calendar

Actor:	Usuario

Descripción:	El usuario solicita al bot programar una actividad en Google Calendar indicando fecha, hora, título y descripción.
Precondición:	El usuario debe haber autorizado Google Calendar desde la página web.
Flujo principal:	1. El usuario escribe una solicitud como “agenda reunión mañana a las 3”. 2. El bot interpreta título, fecha, hora y detalles del evento. 3. El sistema valida que la fecha y hora sean válidas. 4. El bot crea el evento mediante la API de Google Calendar. 5. El bot confirma al usuario que el evento fue programado.
Postcondición:	El evento queda registrado en el calendario conectado del usuario.
Flujo alternativo:	Si falta fecha, hora o permiso de calendario, el bot solicita el dato faltante o envía al usuario a conectar la cuenta.

CU07: Consultar eventos del calendario

Actor:	Usuario
Descripción:	El usuario pide al bot revisar sus próximos eventos o consultar si tiene algo programado en una fecha específica.
Precondición:	El usuario debe tener Google Calendar conectado y eventos accesibles.
Flujo principal:	1. El usuario solicita ver su agenda del día, semana o una fecha concreta. 2. El bot interpreta el rango de fechas solicitado. 3. El sistema consulta los eventos en Google Calendar. 4. El bot ordena y resume los eventos encontrados. 5. El bot envía la agenda al chat.
Postcondición:	El usuario recibe un resumen de sus eventos pendientes.
Flujo alternativo:	Si no existen eventos en el rango solicitado, el bot informa que no hay actividades programadas.

CU08: Revisar cambios pendientes en GitHub

Actor:	Usuario / Administrador
Descripción:	El usuario solicita al bot consultar si un repositorio tiene issues, pull requests, commits recientes o cambios pendientes.
Precondición:	El repositorio debe existir y el usuario debe haber conectado GitHub si necesita información privada.
Flujo principal:	1. El usuario indica el repositorio o usa uno configurado por defecto. 2. El bot valida el nombre del repositorio y el tipo de consulta. 3. El sistema consulta la API de GitHub. 4. El bot resume issues, pull requests o commits pendientes. 5. El bot envía el resumen con enlaces o datos principales.
Postcondición:	El usuario conoce el estado actual del repositorio consultado.
Flujo alternativo:	Si el repositorio es privado o se supera el límite de la API, el bot informa que se requiere token o esperar a que se restablezca la cuota.

CU09: Consultar información de YouTube

Actor:	Usuario
Descripción:	El usuario pide buscar un video, consultar datos de un canal o resumir información básica disponible de YouTube.
Precondición:	El bot debe tener acceso a internet y, si usa YouTube Data API, una clave configurada.
Flujo principal:	1. El usuario envía una búsqueda o enlace de YouTube. 2. El bot detecta si es búsqueda, video o canal. 3. El sistema consulta la información disponible mediante API o servicio configurado. 4. El bot organiza título, canal, enlace y datos relevantes. 5. El bot envía el resultado al usuario.
Postcondición:	El usuario recibe información útil del contenido de YouTube solicitado.

Flujo alternativo:	Si la cuota diaria de YouTube se agota, el bot informa la limitación y sugiere intentar después.

CU10: Guardar preferencias y memoria persistente

Actor:	Usuario
Descripción:	El sistema guarda datos útiles del usuario, preferencias y configuración para mantener continuidad entre sesiones.
Precondición:	La base de datos Neon PostgreSQL debe estar configurada y disponible.
Flujo principal:	1. El usuario interactúa con el bot o actualiza una preferencia. 2. El bot identifica la información que debe conservarse. 3. El sistema valida que no sea información sensible innecesaria. 4. El bot guarda o actualiza el dato en Neon. 5. En futuras conversaciones, el bot consulta esos datos para responder mejor.
Postcondición:	Las preferencias quedan almacenadas y disponibles para futuras interacciones.
Flujo alternativo:	Si Neon no responde, el bot usa memoria temporal y avisa que la persistencia no está disponible.

CU11: Administrar configuración y despliegue

Actor:	Administrador
Descripción:	El administrador configura variables, permisos, integraciones y despliegue en Railway para mantener el bot y la página funcionando.
Precondición:	El proyecto debe estar listo para despliegue y el administrador debe tener permisos.
Flujo principal:	1. El administrador conecta el proyecto a Railway. 2. Configura variables como BOT_TOKEN, GROQ_API_KEY, DATABASE_URL y claves de APIs externas. 3. Configura la URL pública de la página para login y autorización.

	4. Railway compila y ejecuta la aplicación. 5. El administrador revisa logs y prueba la página y el bot desde Telegram.
Postcondición:	La página y el bot quedan disponibles en línea para los usuarios.
Flujo alternativo:	Si falta una variable de entorno o falla el arranque, Railway muestra logs y el administrador corrige la configuración.

Viabilidad gratis o de bajo costo

Sí se puede hacer una versión gratis o casi gratis para entrega académica, siempre que el uso sea pequeño y se controlen las cuotas. Railway ofrece prueba inicial y crédito mensual limitado; Neon tiene plan gratuito para Postgres pequeño; Google Calendar funciona sin cobro estándar dentro de sus cuotas; GitHub permite consultas gratis con límites de tasa; YouTube Data API funciona por cuota diaria. Para evitar bloqueos, conviene usar tokens oficiales, caché, pocos llamados repetidos y mensajes claros cuando una cuota se agote.

Recomendación de alcance para la primera versión

- Versión mínima: página web simple, verificación con Telegram, /start, /menu, conversación con IA, juegos/utilidades y despliegue en Railway.
- Versión con base de datos: guardar usuarios verificados, permisos conectados, preferencias, historial resumido y logs principales en Neon PostgreSQL.
- Versión con integraciones: Calendar para crear/consultar eventos, GitHub para revisar repositorios y YouTube para consultar videos o canales.
- Evitar para la entrega inicial: módulos ajenos al proyecto Telegram Bot o funcionalidades de otros juegos/proyectos del workspace.